

Feed-Forward Networks and Backpropagation

From parameter counting to multi-GPU training · theory, intuition, NumPy code and figures · Section 02 of

Deep-Learning-and-Machine-Learning-Models

A **feed-forward neural network** (FFNN, or multi-layer perceptron) is a chain of simple functions: each layer multiplies its input by a weight matrix, adds a bias and applies a non-linearity. **Backpropagation** is nothing more than the chain rule applied to that chain, organised so every layer's gradient is computed once and reused. This guide follows the six notebooks of this folder: **(1)** counting parameters, **(2)** the matrix-form forward pass, **(3)** gradients at the output layer, **(4)** gradients through hidden layers, **(5)** training a network end to end, and **(6)** memory footprint and multi-GPU training. Every number quoted below was produced by the code in the notebooks.

Conventions. A batch is a matrix $\mathbf{X}$ with one row per sample, shape (N, d) . Layers are numbered $1 \dots L$ with $A[0] = \mathbf{X}$; $W[l]$ has shape (n_{l-1}, n_l) and $b[l]$ has shape (n_l) . The loss is the *mean* over the batch, so the factor $1/N$ lives in the output error signal.

1 Architecture and Parameter Counting

A fully-connected layer connects **every** neuron of the previous layer to **every** neuron of the current one, and each neuron owns one bias. Hence a layer with n_{l-1} inputs and n_l outputs has

$$\#params_l = n_{l-1} n_l + n_l, \quad \#params = \sum_{l=1}^L (n_{l-1} n_l + n_l)$$

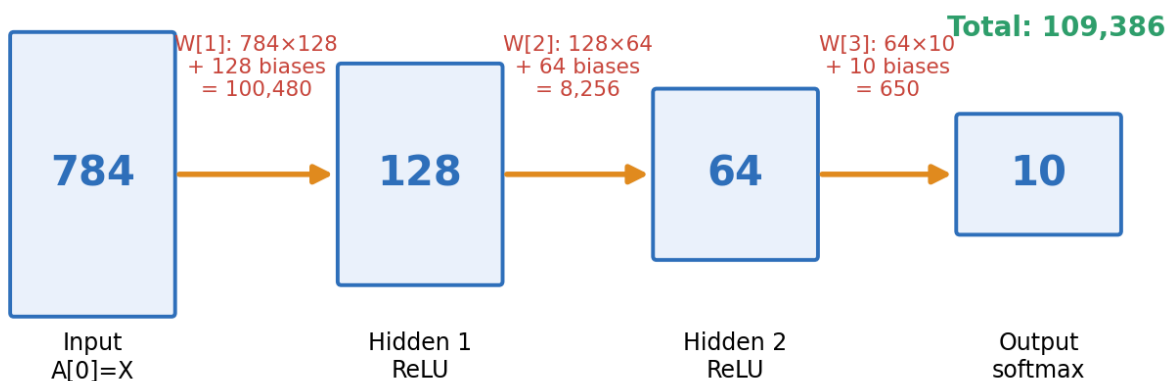


Figure 1 - A 784-128-64-10 classifier (MNIST-sized). The three weight matrices dominate the parameter count.

```
def count_params(sizes):
    rows, total = [], 0
    for l, (n_in, n_out) in enumerate(zip(sizes[:-1], sizes[1:]), start=1):
        w, b = n_in * n_out, n_out
        rows.append(dict(layer=l, shape=(n_in, n_out), weights=w, biases=b, total=w + b))
        total += w + b
    return rows, total
```

```
rows, total = count_params([784, 128, 64, 10])    # -> total = 109,386
```

Layer	W shape	Weights	Biases	Total	Share
1	784 × 128	100,352	128	100,480	91.9 %
2	128 × 64	8,192	64	8,256	7.5 %
3	64 × 10	640	10	650	0.6 %
Sum		109,184	202	109,386	100 %

Intuition

With a large input, the **first layer dominates**: 784 inputs × 128 neurons is already 92 % of the model. Memory in fp32 is 4 bytes × parameters (here 0.44 MB). Beyond parameters, a forward pass costs about $2 \times (\#weights)$ FLOPs per sample (one multiply + one add per weight) and the activations that backprop must store scale with batch size × $\sum n_l$. Compare widths: 784-512-10 has 407,050 parameters, 784-128×4-10 only 151,306 - depth is cheap when the input layer is the bottleneck.

2 The Matrix-Form Forward Pass

One neuron computes $z = w \cdot x + b$ and then $a = g(z)$. Doing that with Python loops over samples, neurons and inputs is correct but slow. Stacking all neurons of a layer as the columns of W , and all samples as the rows of A , collapses three nested loops into **one matrix product**:

$$Z^{[l]} = A^{[l-1]} W^{[l]} + b^{[l]}, \quad A^{[l]} = g(Z^{[l]}), \quad A^{[0]} = X$$

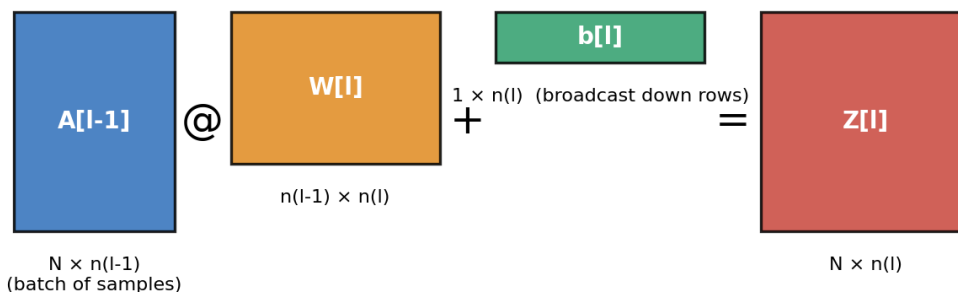


Figure 2 - Shape bookkeeping. The bias row is broadcast down all N rows; N never appears in the parameters.

Activations. Without a non-linearity, two layers collapse into one linear map, $W_2(W_1x) = (W_2W_1)x$. Sigmoid squashes to $(0, 1)$ but saturates (derivative ≤ 0.25); tanh is zero-centred; ReLU is cheap and has derivative 1 for $z > 0$. The last layer uses **softmax** to produce class probabilities:

$$\text{softmax}(z)_k = \frac{e^{z_k - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}$$

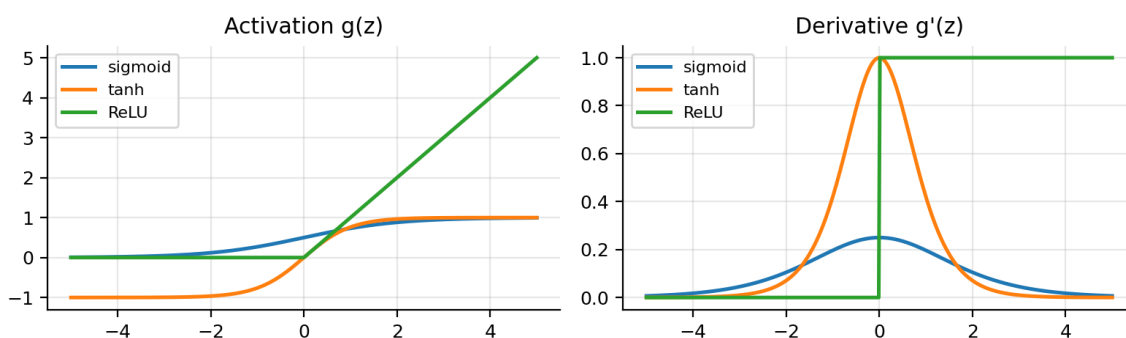


Figure 3 - Common activations (left) and their derivatives (right). The derivative decides how much gradient survives backprop.

```
def forward(X, params):
    A, L = X, len(params)
    cache = [(None, X)]
    for l, (W, b) in enumerate(params, start=1):
        Z = A @ W + b
        A = softmax(Z) if l == L else relu(Z)
        cache.append((Z, A))
    return A, cache

# layer 1: A(32, 784) @ W(784, 128) + b(128,) -> Z(32, 128) --ReLU--> A(32, 128)
# layer 3: A(32, 64) @ W(64, 10) + b(10,) -> Z(32, 10) --softmax--> A(32, 10)
```

Why it matters

In the notebook the loop version took **201 ms** and the matrix version **0.145 ms** on the same 32×256 @ 256×64 problem - a $\sim 1,400\times$ speed-up, and the same expression runs unchanged on GPUs. Softmax subtracts the row maximum: for logits $[1000, 1001, 1002]$ the naive formula gives NaN, the stable one gives $[0.090, 0.245, 0.665]$. **Hand check:** $x = [1, 0.5]$, $W_1 = \begin{bmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \end{bmatrix}$, $b_1 = [0.1, -0.1]$ gives $Z_1 = [0.5, -0.35]$, ReLU $\rightarrow [0.5, 0]$; with $W_2 = [0.6, -0.2]^T$, $b_2 = 0.05$ we get $z = 0.35$ and $\hat{y} = \sigma(0.35) = 0.5866$.

3 Backpropagation for the Output Layer

Training needs the gradient of the loss with respect to every weight. Define the **error signal** $\delta^{[l]} = \partial L / \partial Z^{[l]}$ - how much the loss would change if the pre-activation of layer l moved. Once δ is known, the parameter gradients of a dense layer follow from the product rule, because $Z = A_{\text{prev}} W + b$:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = (A^{[l-1]})^\top \delta^{[l]}, \quad \frac{\partial \mathcal{L}}{\partial b^{[l]}} = \sum_n \delta_{n, :}^{[l]}, \quad \frac{\partial \mathcal{L}}{\partial A^{[l-1]}} = \delta^{[l]} (W^{[l]})^\top$$

For a softmax output with cross-entropy loss $L = -(1/N) \sum y \log p$, the softmax Jacobian $\partial p_i / \partial z_j = p_i (\delta_{ij} - p_j)$ and the loss derivative $-y/p$ combine so that almost everything cancels:

$$\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial Z^{[L]}} = \frac{1}{N} (P - Y)$$

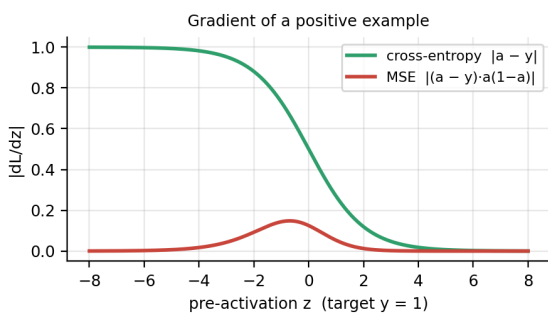
Intuition: the error at each logit is simply *predicted probability minus truth*. A confident correct prediction gives $\delta \approx 0$; a confident wrong one pushes hard.

```
delta = (P - Y) / N          # (N, K) error signal at the output
dW     = A_prev.T @ delta    # (d, N) @ (N, K) -> (d, K)
db     = delta.sum(axis=0)   # (K,)
dA_prev = delta @ W.T        # (N, K) @ (K, d) -> (N, d) flows to the layer below
```

Always gradient-check. Central finite differences give an independent estimate of every derivative, and a relative error below 10^{-6} means the analytic code is right:

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta_i + \epsilon) - \mathcal{L}(\theta_i - \epsilon)}{2\epsilon}$$

```
def numerical_grad(f, x, eps=1e-6):
    g = np.zeros_like(x); it = np.nditer(x, flags=["multi_index"])
    for _ in it:
        i = it.multi_index; old = x[i]
        x[i] = old + eps; fp = f(); x[i] = old - eps; fm = f(); x[i] = old
        g[i] = (fp - fm) / (2 * eps)
    return g
# result: dW 2.0e-10 db 8.0e-10 dA_prev 6.0e-10 (relative error)
```



Why cross-entropy beats MSE with sigmoid. For a positive example the MSE gradient is $(a-1) \cdot a(1-a)$. When the neuron is confidently wrong ($z \rightarrow -\infty$) the sigmoid is flat, $a(1-a) \rightarrow 0$, and learning stalls. Cross-entropy's gradient $a-1 \rightarrow -1$ stays large exactly when the mistake is worst. This is why the pairs softmax+CE, sigmoid+BCE and identity+MSE all give the clean $\delta = (A-Y)/N$.

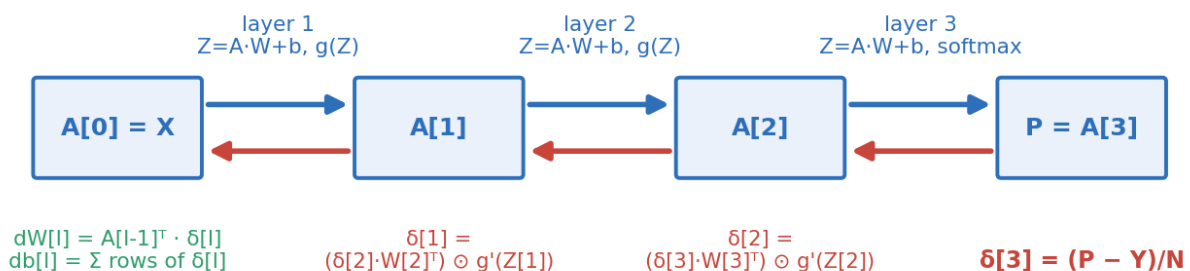
Figure 4 - gradient magnitude of the two losses.

4 Backpropagation Through Hidden Layers

A hidden layer's output $A^{[l-1]}$ influences the loss only through the next layer. So the error of layer $l-1$ is the error of layer l , **sent back through the weights** $W^{[l]}$ and then multiplied by the local sensitivity $g'(Z)$ of the activation:

$$\delta^{[l]} = (\delta^{[l+1]} (W^{[l+1]})^\top) \odot g'(Z^{[l]})$$

forward sweep (store Z, A) →



← backward sweep (reuse stored Z, A)

Figure 5 - One forward sweep stores Z and A; one backward sweep turns $\delta[3]$ into $\delta[2]$, $\delta[1]$ and every dW , db on the way.

```
def backward(Y, params, cache, gprime):
    N, L = Y.shape[0], len(params); grads = [None] * L
    delta = (cache[-1][1] - Y) / N # output layer
    for l in range(L, 0, -1):
        A_prev = cache[l - 1][1]; W, _ = params[l - 1]
        grads[l - 1] = (A_prev.T @ delta, delta.sum(axis=0)) # dW, db
        if l > 1:
            delta = (delta @ W.T) * gprime(cache[l - 1][0]) # send back, gate by g'(Z)
    return grads
# gradient check on a 4-layer tanh network: every dW, db has relative error ~1e-9
```

Backprop is **reverse-mode automatic differentiation** specialised to a chain. The backward sweep needs two matrix products per layer (one for dW, one for δ), so it costs about **twice the forward pass**, and it needs every Z and A from the forward pass - which is why activation memory exists (Section 6).

Vanishing gradients, measured. Each backward step multiplies by $g'(z)$. For sigmoid $g' \leq 0.25$, so the signal shrinks geometrically with depth. In a 10-hidden-layer network the gradient norm of the first layer was 1.7×10^{-7} with sigmoid (vs 0.92 at the last layer), **0.10** with tanh and **0.90** with ReLU + He initialisation, whose derivative is exactly 1 on the active side. Initial scale matters too: He uses $\text{Var}(W) = 2/n_{\text{in}}$ to keep activations and gradients at a stable size.

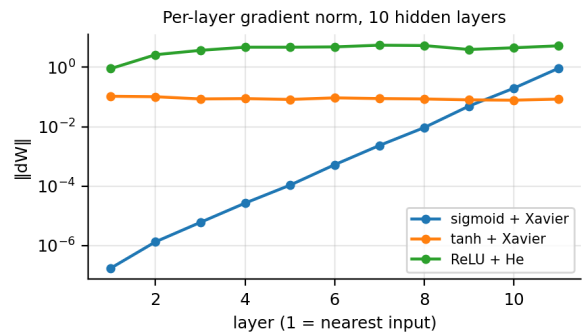


Figure 6 - per-layer gradient norm.

Worked example: one complete backprop step by hand

Take the 2-2-1 network of Section 2 with a ReLU hidden layer, a sigmoid output and binary cross-entropy, one sample $x = [1, 0.5]$ with target $y = 1$. Forward: $Z[1] = [0.5, -0.35]$, $A[1] = [0.5, 0]$, $Z[2] = 0.35$, $\hat{y} = 0.5866$, loss = $-\ln(0.5866) = \mathbf{0.5334}$. Backward:

Step	Computation	Result
Output error	$\delta[2] = \hat{y} - y = 0.5866 - 1$	-0.4134
Output weights	$dW[2] = A[1]^T \cdot \delta[2]$; $db[2] = \delta[2]$	$[-0.2067, 0]^T$; -0.4134
Send back	$\delta[2] \cdot W[2]^T = -0.4134 \cdot [0.6, -0.2]$	$[-0.2480, 0.0827]$
Gate by $g'(Z[1])$	$g'([0.5, -0.35]) = [1, 0]$ (ReLU)	$\delta[1] = [-0.2480, 0]$
Hidden weights	$dW[1] = x^T \cdot \delta[1]$; $db[1] = \delta[1]$	$[-0.2480, 0]$; $[-0.1240, 0]$; $[-0.2480, 0]$
SGD step ($\eta = 0.5$)	$\theta \leftarrow \theta - \eta \cdot \nabla \theta$, then forward again	$\hat{y}: 0.5866 \rightarrow 0.6910$; loss: $0.5334 \rightarrow \mathbf{0.3697}$

Notice the second hidden neuron: its pre-activation was negative, so ReLU's derivative is 0 and **its whole column of $W[1]$ receives zero gradient** for this sample - the 'dying ReLU' effect in miniature. One update already cut the loss by 31 %.

5 Training a Small Network

Forward, loss and backward give gradients; an **optimiser** turns them into updates. **Mini-batch SGD** shuffles the data each epoch, processes small batches and steps against the gradient: $\theta \leftarrow \theta - \eta g$. **Momentum** keeps a running velocity that smooths noisy mini-batch gradients and speeds up travel along consistent directions:

$$v \leftarrow \mu v - \eta g, \quad \theta \leftarrow \theta + v$$

```
class MLP:
    # forward(), backward() as in Sections 2-4, plus:
    def step(self, gW, gb, lr, momentum=0.0):
        for l in range(len(self.W)):
            self.vW[l] = momentum * self.vW[l] - lr * gW[l]; self.W[l] += self.vW[l]
            self.vb[l] = momentum * self.vb[l] - lr * gb[l]; self.b[l] += self.vb[l]

    def fit(self, X, Y, Xte, Yte, epochs, batch, lr, momentum=0.0):
        for ep in range(epochs):
            idx = rng.permutation(len(X)) # shuffle every epoch
            for s in range(0, len(X), batch):
                j = idx[s:s + batch]
                self.forward(X[j]); gW, gb = self.backward(Y[j]); self.step(gW, gb, lr, momentum)
```

A single linear layer cannot solve XOR or the spiral, but one hidden layer can bend the decision boundary: the network reaches **100 % train and test accuracy** on the spiral and solves XOR to a loss of 2×10^{-4} .

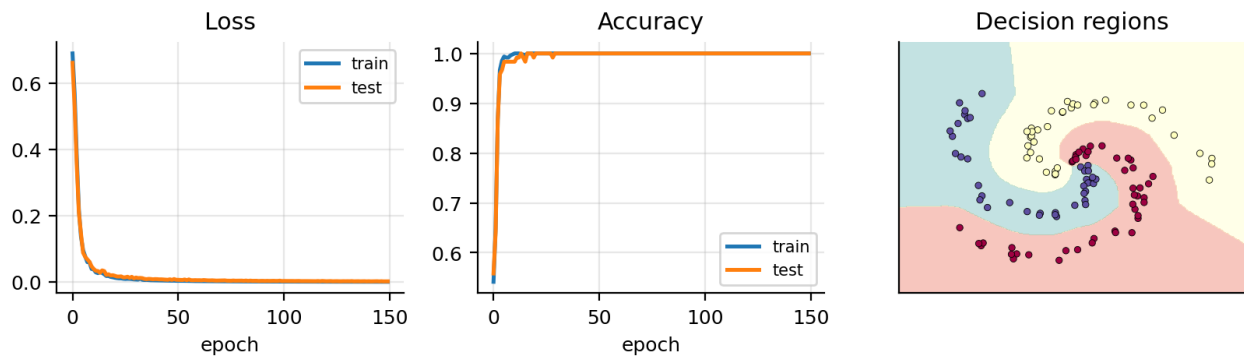
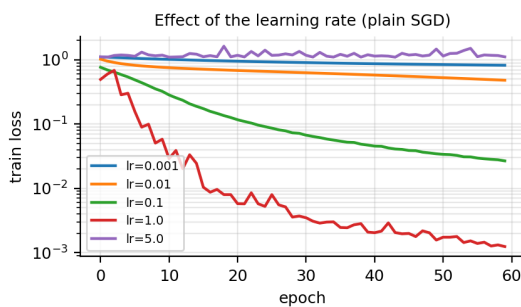


Figure 7 - A 2-64-64-3 ReLU network (η 0.05, momentum 0.9) on a 3-arm spiral: loss and accuracy curves, and the learned decision regions.



The learning rate is the most sensitive knob. With plain SGD for 60 epochs: $\eta = 0.001$ is far too slow (loss 0.82, 52 % test accuracy), $\eta = 0.01$ learns slowly (85 %), $\eta = 0.1$ and 1.0 work (99 % / 100 %), and $\eta = 5$ **overshoots** - the loss bounces and the model is at chance (33 %). Practical recipe: sweep η on a log scale, watch the loss curve, then add momentum, weight decay and a decay schedule.

Figure 8 - training loss for five learning rates.

6 Memory Footprint and Multi-GPU Training

Before launching a run, you can predict GPU memory from the architecture. With fp32 weights and the Adam optimiser each parameter needs 4 bytes (weight) + 4 (gradient) + 8 (Adam's two moment estimates), i.e. **16 bytes per parameter**, plus activations stored for backprop:

$$M \approx 16P + 4B \left(n_0 + 2 \sum_{l=1}^L n_l \right) \text{ bytes}$$

The notebook allocates real float32 arrays for a 784-4096-4096-10 MLP at batch 256 and the estimate matches nbytes exactly: weights 80.15 MB, gradients 80.15 MB, Adam states 160.30 MB, activations 17.60 MB - **338 MB** in total. Parameters are fixed, but activations grow linearly with batch size. At LLM scale the 16 B/param rule alone gives 16 GB for 1B parameters, **112 GB for 7B** and 1,120 GB for 70B - more than one 80 GB GPU.

Technique	What is split	What is communicated	Solves
Data parallel (DDP)	the batch	gradients (all-reduce)	throughput
ZeRO-1/2/3 · FSDP	optimizer / + grads / + weights	shards gathered on demand	per-GPU memory
Pipeline parallel	layers	activations between stages	model too big for one GPU
Tensor parallel	each matmul	partial results	huge layers

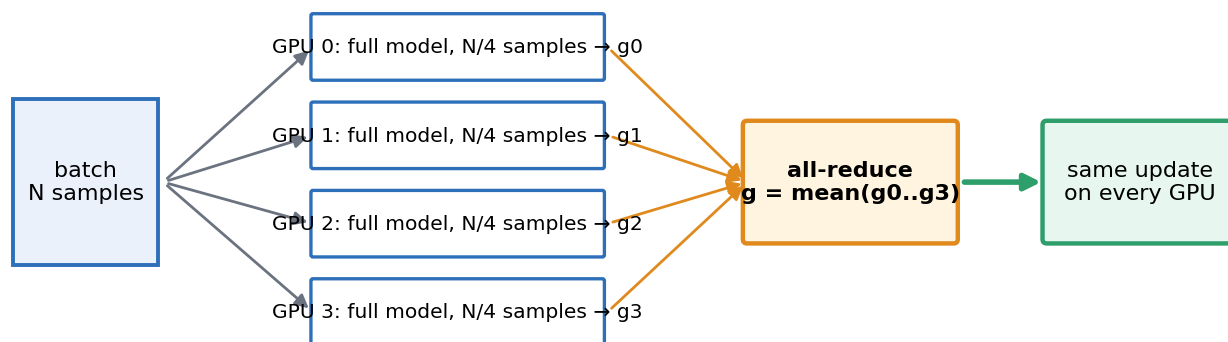


Figure 9 - Data parallelism: identical model copies, different data slices, one all-reduce, identical updates.

Data parallelism is exact. Every GPU holds a full model copy, computes gradients on N/K samples, and the gradients are averaged with an all-reduce. Because the loss is a mean, the weighted average of shard gradients equals the full-batch gradient - the notebook confirms a maximum difference of 5.6×10^{-17} . A **ring all-reduce** sends only $2(K-1)/K$ of the gradient size per GPU, almost independent of K .

ZeRO / FSDP remove the redundancy: for a 7B model on $K = 8$ GPUs, plain DDP needs 112 GB per GPU, ZeRO-1 63 GB, ZeRO-2 38.5 GB and ZeRO-3 only **14 GB**.

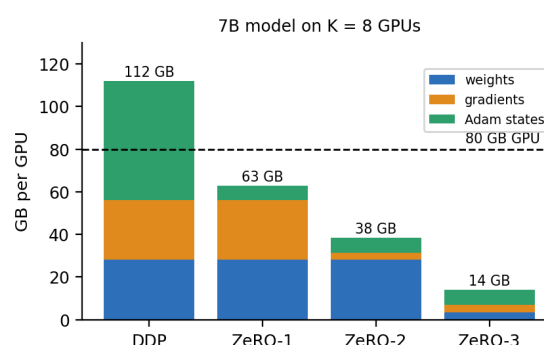


Figure 10 - per-GPU memory by sharding stage.

```

shards = np.array_split(np.arange(N), K)
g_local = [flat(grads_for(params, X[s], Y[s])) for s in shards]
g_avg = sum(len(s) / N * g for s, g in zip(shards, g_local))
assert np.allclose(flat(grads_for(params, X, Y)), g_avg)
  
```

```

# each "GPU" gets N/K samples
# independent backward passes
# all-reduce (weighted mean)
# == single-device gradient
  
```

Debugging checklist

Symptom	Likely cause	Check
Loss is NaN after a few steps	learning rate too high, $\log(0)$, unstable softmax	lower η ; subtract row max; add $1e-12$ inside log
Loss stuck at $\ln(K)$ (e.g. 1.0986 for 3 classes)	dead network: zero/tiny init, bad η , labels not shuffled	He init; print $\ dW\ $ per layer; shuffle each epoch
Gradient check error $> 1e-3$	shape/transpose slip, missing $1/N$, wrong g'	check layer by layer; use tanh (smooth) for the check
Train accuracy high, test low	over-fitting	weight decay, more data, smaller net, early stopping
Out-of-memory on GPU	batch too large, optimizer states, activations	estimate 16 B/param + activations; shrink batch; ZeRO/FSDP; checkpointing
Multi-GPU run diverges from 1-GPU run	unequal shard sizes averaged unweighted	weight shard gradients by n_k/N

Computational-graph view. Every operation in the forward pass (matmul, add, activation, loss) is a node that knows its own local derivative. Backprop walks the graph in reverse and multiplies local derivatives along each path, summing where paths merge. The formulas in this guide are that process written once for dense layers; frameworks such as PyTorch automate it for any graph, but the cost structure - one stored forward pass, a backward pass of about twice the compute, memory proportional to stored activations - is exactly what you computed here.

Cheat-sheet

Concept	Formula	Shape / note
Parameters of a layer	$n_{in} \cdot n_{out} + n_{out}$	109,386 for 784-128-64-10
Forward	$Z = A \cdot W + b ; A = g(Z)$	$(N, n_{in}) @ (n_{in}, n_{out})$
Output error (softmax+CE)	$\delta[L] = (P - Y)/N$	(N, K)
Weight / bias gradient	$dW = A[l-1]^T \delta[l] ; db = \sum \text{rows } \delta[l]$	same shape as W, b
Hidden error	$\delta[l] = (\delta[l+1] \cdot W[l+1]^T) \odot g'(Z[l])$	uses stored Z[l]
SGD + momentum	$v \leftarrow \mu v - \eta g ; \theta \leftarrow \theta + v$	tune η first
Training memory (fp32, Adam)	16 B/param + activations	ZeRO-3: 16P/K per GPU

Where to go next

Run the six notebooks in order (each has exercises at the end), then reuse the MLP class as the building block for the next section of the repository - word embeddings, RNNs and the Transformer all reuse these same ideas: matrix forward passes, local gradients chained backwards, and memory/parallelism arithmetic.